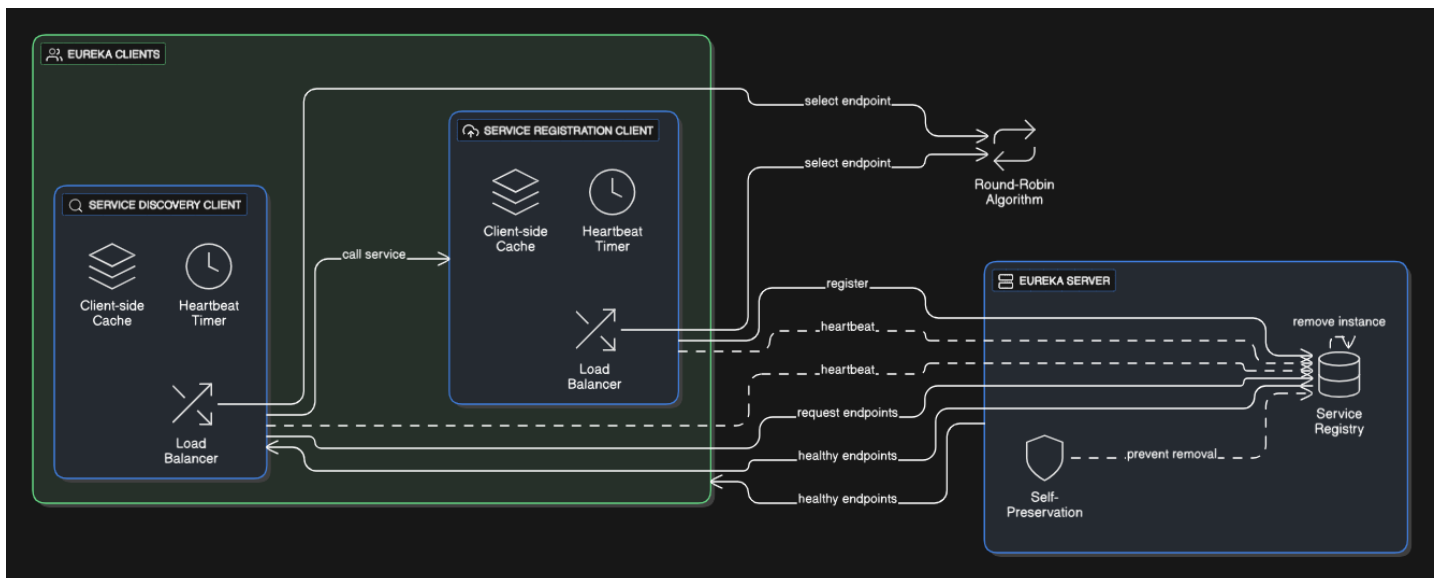
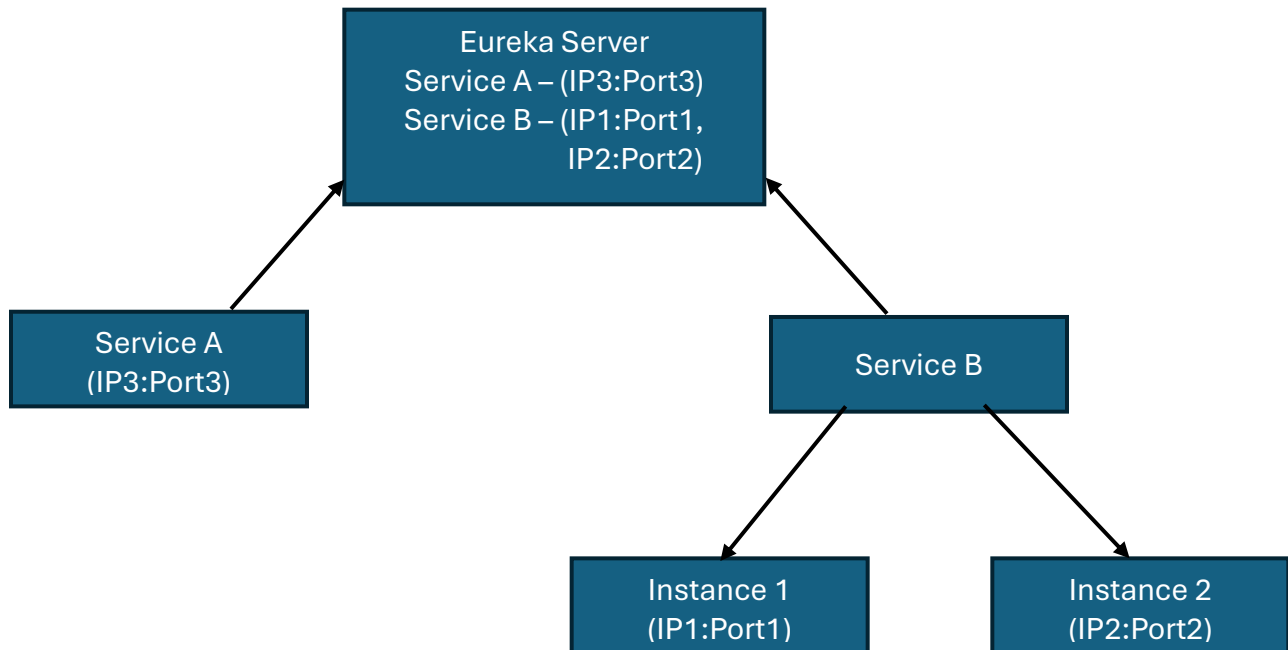


Service Discovery

When a service wants to communicate with another service it faces the problem of dynamic IP and port configuration for various instances of the same service. The Service Discovery concept solves this issue by maintaining a Service Registry which effectively is a mapping between service names and their corresponding IP and ports.



Service discovery is a concept of microservices which solves the problem of one service finding another where each service has different IP addresses and ports

Eureka Server, on the other hand is an implementation of the **Service Discovery** concept.

The components used in most implementations of service discovery are:

1. Server:

- a. The main responsibility of the server is to maintain a **service registry** which keeps a record of all clients registering on the server.
- b. Suppose two clients want to communicate with each other, how will they find each other?
 - i. Client A invokes the Eureka server
 - ii. The Eureka server in turn returns a list of all healthy endpoints of Client B.
 - iii. Client B then chooses the best endpoint based on the client-side load balancing algorithm **[Round-Robin in case of Spring Cloud]**

2. Eureka client:

- a. The client is responsible for registering itself on the eureka server.
- b. The client then sends a heartbeat signal to the eureka server every 30 seconds to indicate that it is alive.
- c. If the server doesn't receive a heartbeat signal for 90 seconds, the service is marked as down.
- d. The client maintains a client-side load balancer which automatically chooses the best endpoint from the list of healthy endpoints returned by the eureka server.

3. Client-side cache:

- a. The client maintains a local copy of the service registry, so that even if the eureka server goes down, the client can still access other services using its cached copy.
- b. This feature is important for the resiliency of the application.
- c. The client-side cache is preserved for 3 mins during which time each client fetches updates from the server every 30 seconds.

4. Self-Preservation:

- a. When many client instances start going down simultaneously the eureka server doesn't remove them from the registry and instead keeps them running. This is being done to ensure that temporary network issues don't make entire system go down.
- b. But what exactly does the word **"Large"** signify. Well, if $> 15\%$ of clients go down together then the server enters self-preservation mode automatically, unless otherwise it is disabled.

5. Load Balancing:

- a. The load-balancing is done at the client side. The client, on receiving the healthy endpoints from the server uses a load balancing algorithm to decide which endpoint to choose.